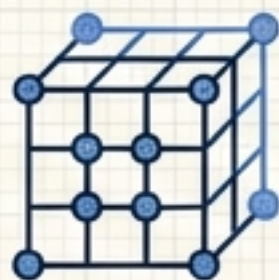
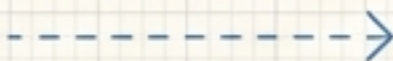


The Mathematical Engine of AI & Machine Learning

# NumPy



42 3.14 7 -25.0 9  
0.001 108 12 56.7 3  
88 3 -4 9.8 22.5  
100 6 0.5 500 11



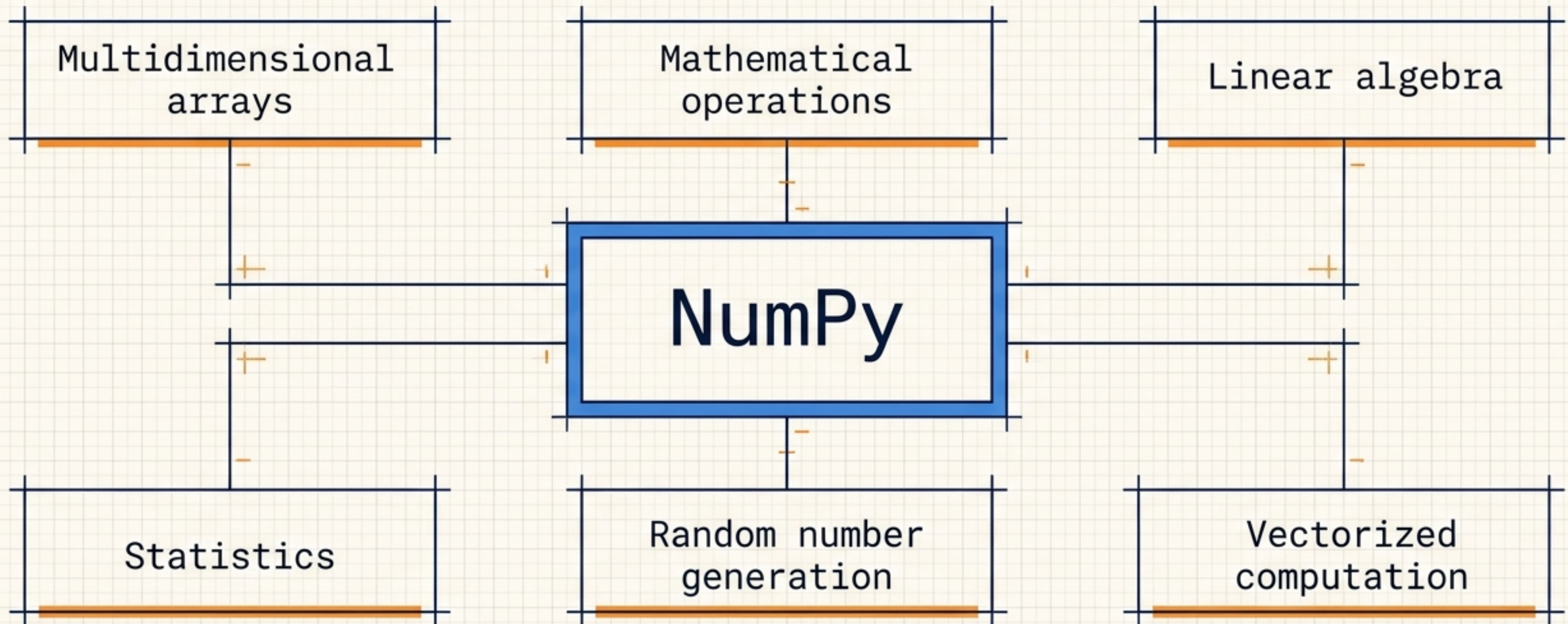
|       |      |      |       |      |
|-------|------|------|-------|------|
| 42    | 3.14 | 7    | -25.0 | 9    |
| 0.001 | 108  | 12   | 56.7  | 3    |
| 88    | -4   | 9.8  | 1     | 22.5 |
| 100   | 6    | 0.5  | 500   | 11   |
| 2     | 77   | 99.9 | 5     | 33   |

ndim=2 shape=(5,5) dtype=float64

*Python lists walked so NumPy arrays could run.*

# The Foundation of Numerical Computing

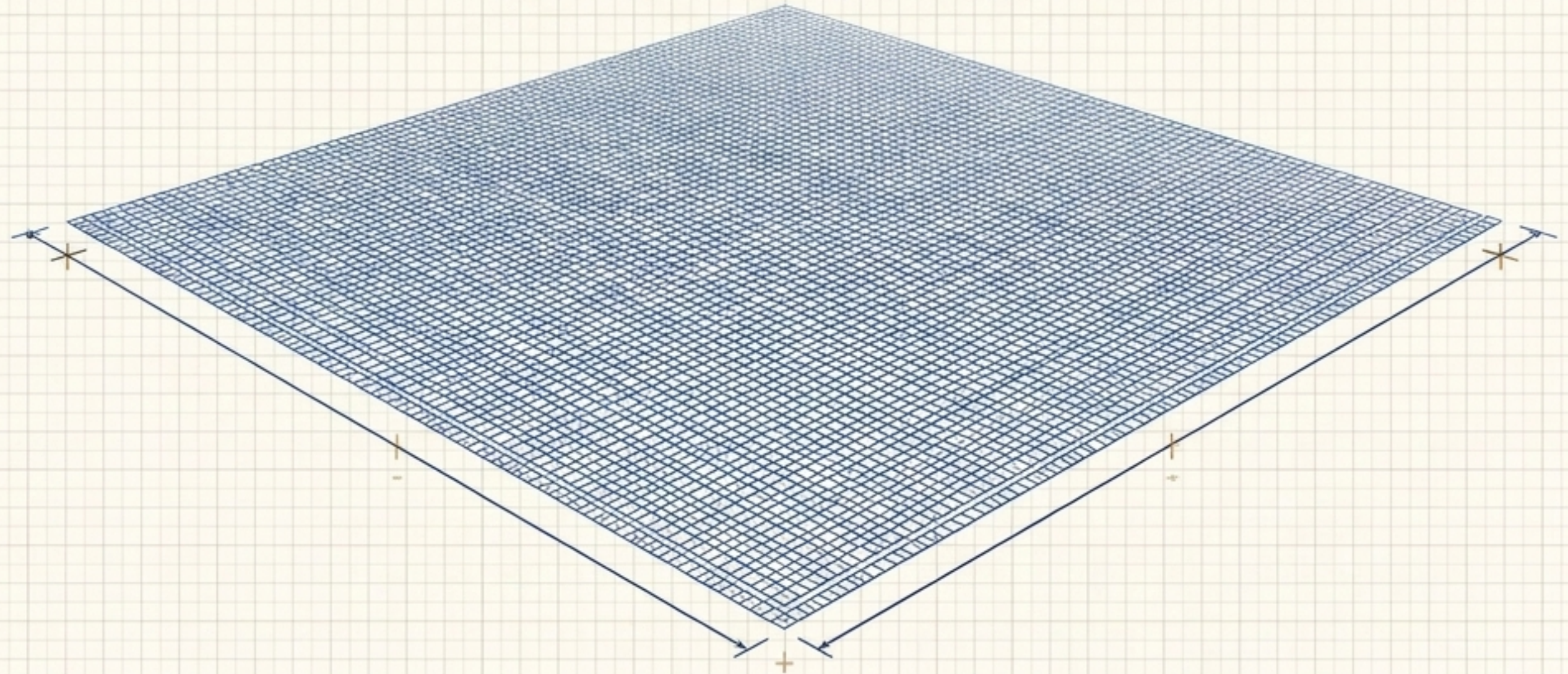
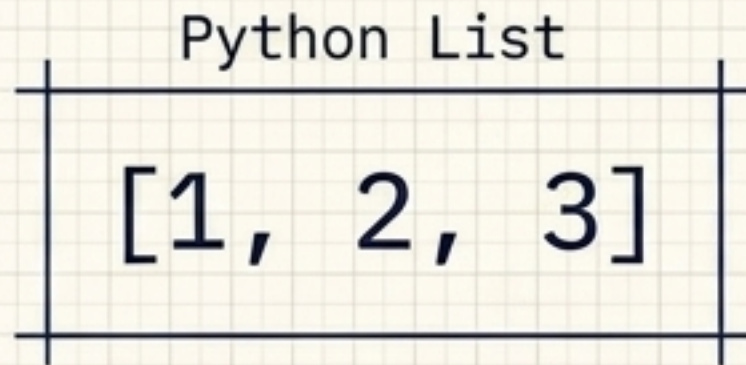
NumPy (Numerical Python) is the specialized, C-backed Python library designed specifically for fast numerical and scientific computing.



# The Scaling Bottleneck

Python lists are highly flexible, but they choke on massive numerical operations. Standard Python operations become incredibly inefficient when scaling to millions of data points.

10,000,000 values

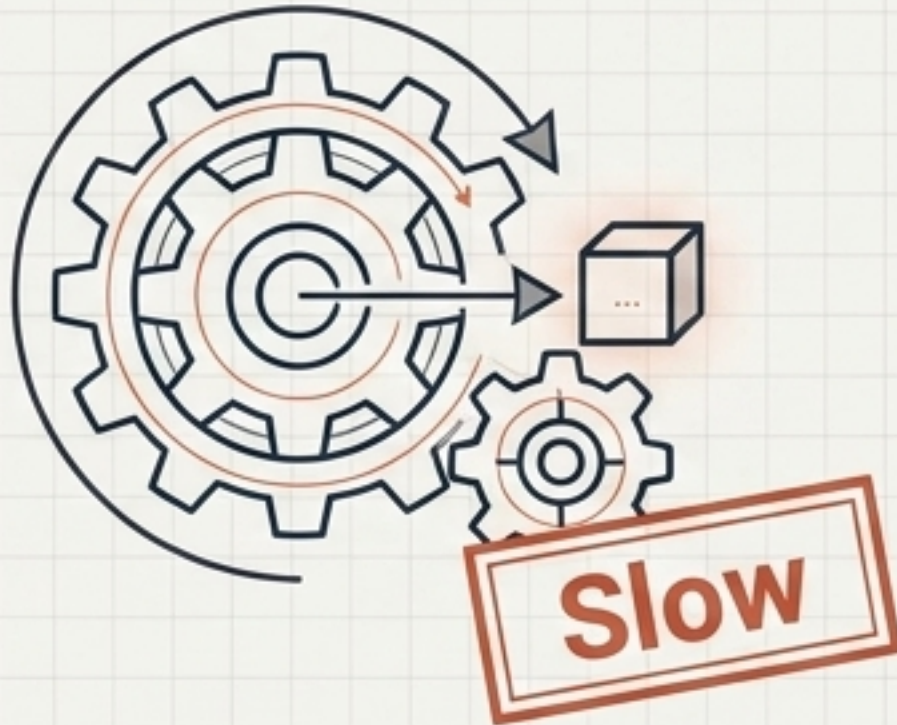


# The Paradigm Shift: Iteration vs. Element-Wise Operations

## Python List

```
[x + 1 for x in list]
```

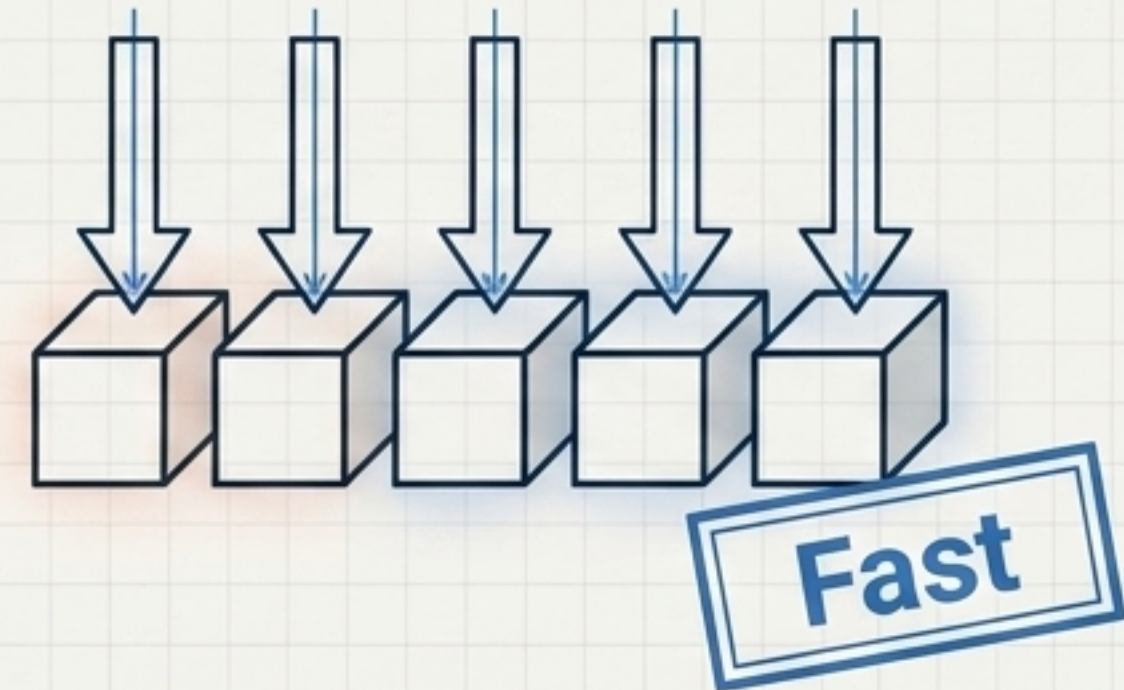
**Mechanic:** Iterates one-by-one.



## NumPy Array

```
array + 1
```

**Mechanic:** Element-by-element simultaneously.



# Anatomy of the ndarray

The ndarray (N-dimensional array) is the foundational data structure of the entire NumPy ecosystem.

Standard  
alias  
convention.

```
import numpy as np
```

```
arr = np.array([1, 2, 3])
```

```
type(arr) # Output: <class 'numpy.ndarray'>
```

The array  
conversion  
function.

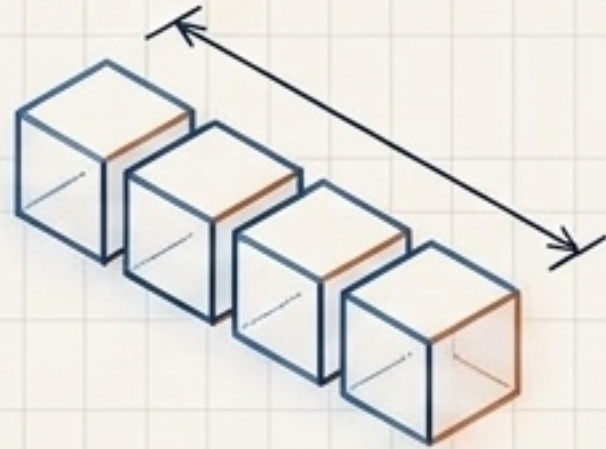
N-dimensional array



# The Geometry of AI Data

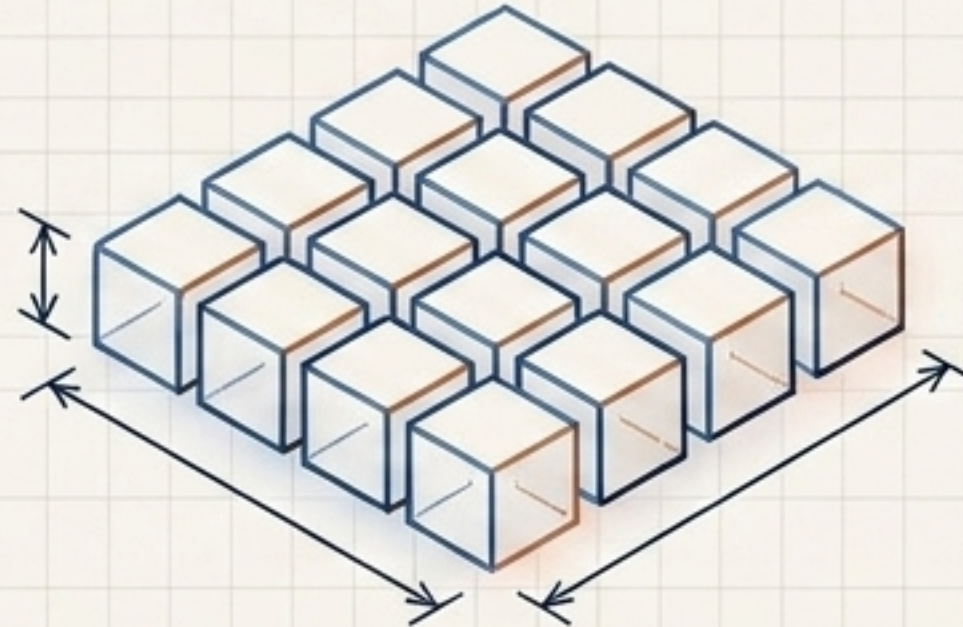
Arrays can hold any number of dimensions. The `.shape` attribute tells us the exact size of each dimension.

1D (Vector)



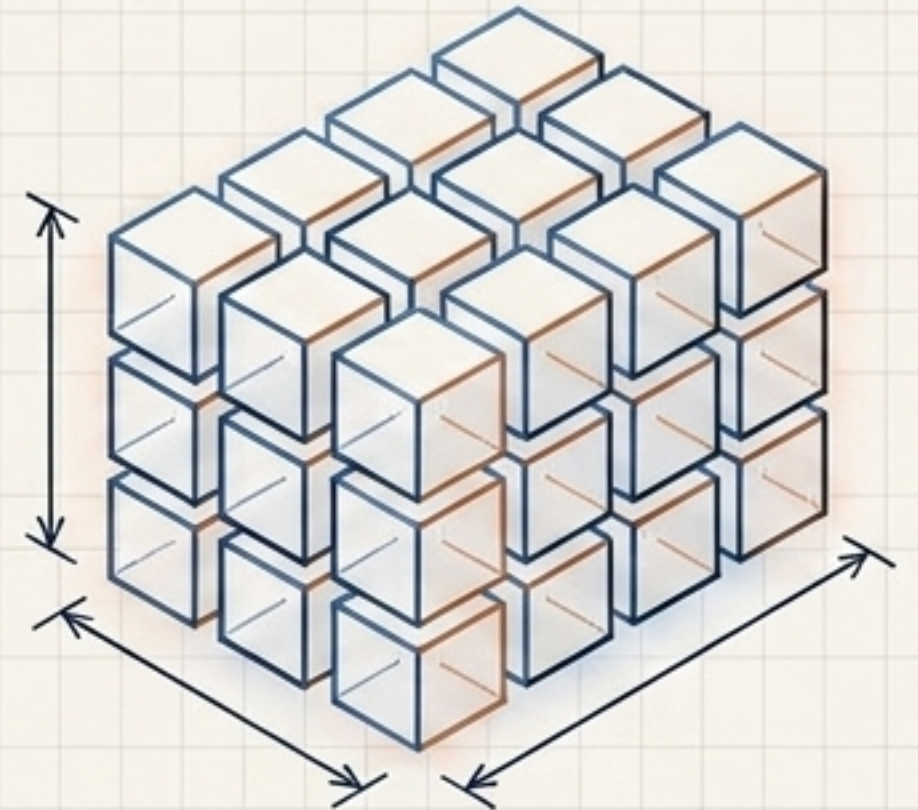
```
.shape: (4,)
```

2D (Matrix)



```
.shape: (3, 4)
```

3D (Tensor)



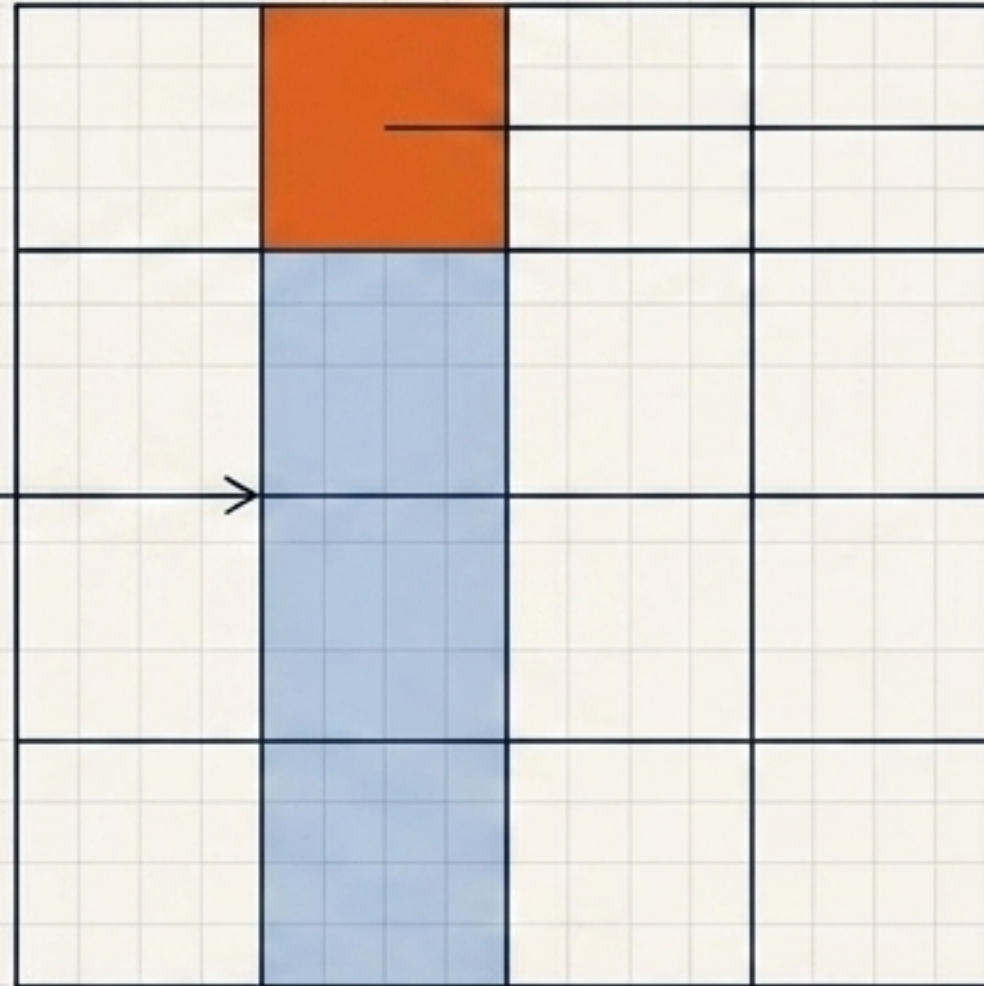
```
.shape: (2, 3, 4)
```

**NOTE:** 3D arrays and above are the critical structures for **Computer Vision** and **Deep Learning**.

# Precision Extraction: Indexing & Slicing

NumPy relies on zero-based indexing to extract precise features and samples from datasets.

`matrix[:, 1]` ○  
Slicing: Take every row  
from column 1.

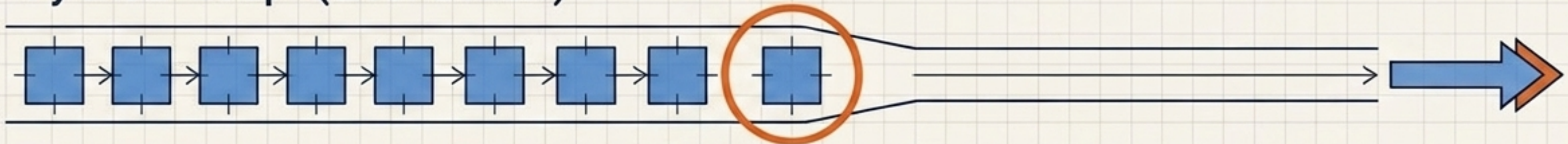


○ `matrix[0, 1]`  
Extracts a single item.

# The Speed Secret: Vectorization

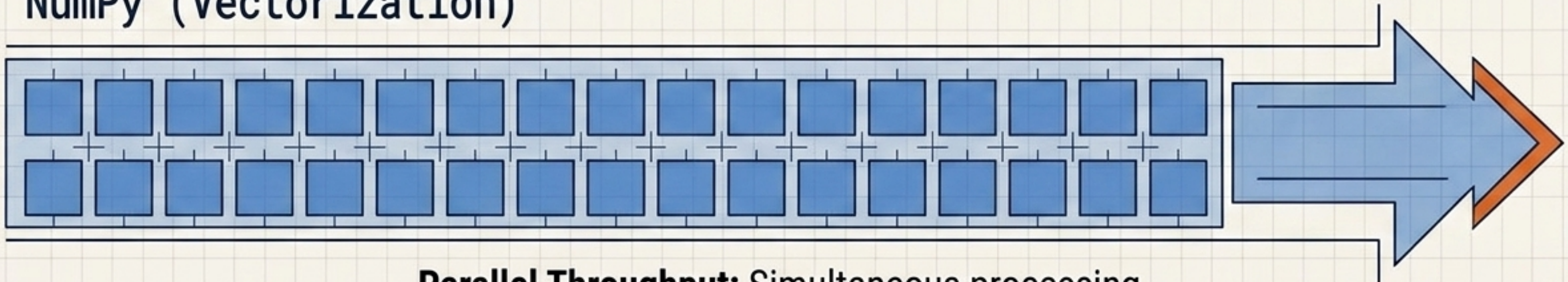
Instead of explicitly writing loops in Python, NumPy passes operations down to highly optimized underlying C implementations to process data in parallel.

## Python Loop (Iteration)



**Sequential Bottleneck:** One-by-one processing.

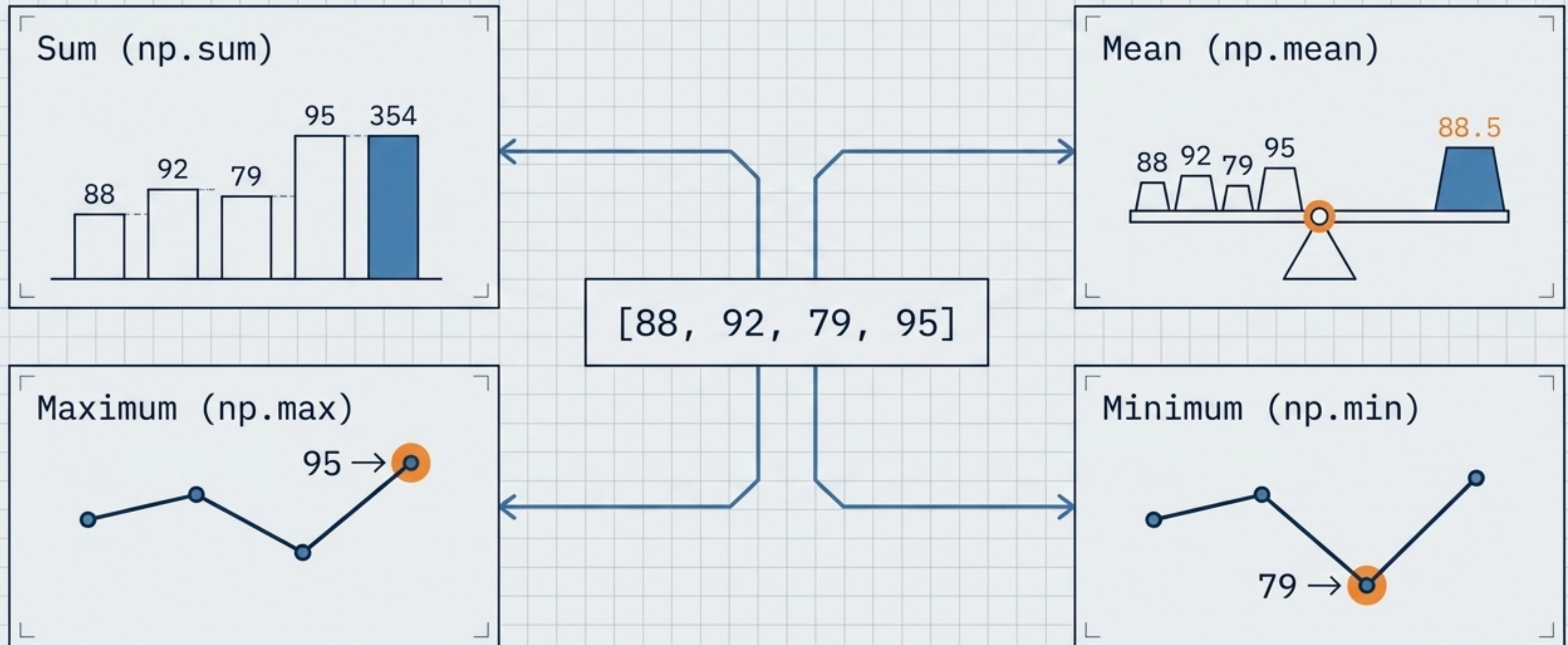
## NumPy (Vectorization)



**Parallel Throughput:** Simultaneous processing.

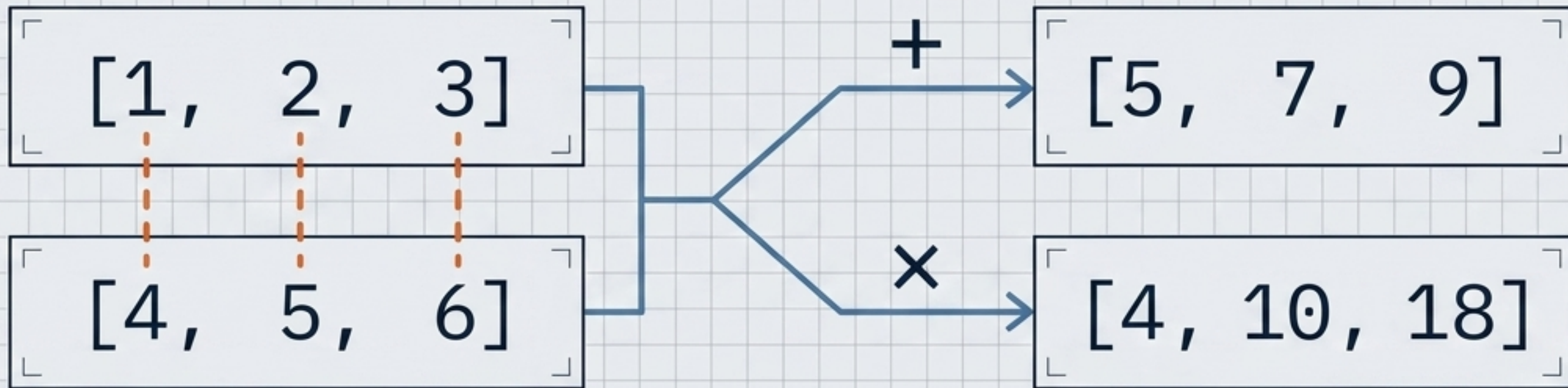
# The Mathematical Arsenal: Aggregation

NumPy instantly collapses massive arrays into single, meaningful statistical metrics.



# Element-Wise Mathematics

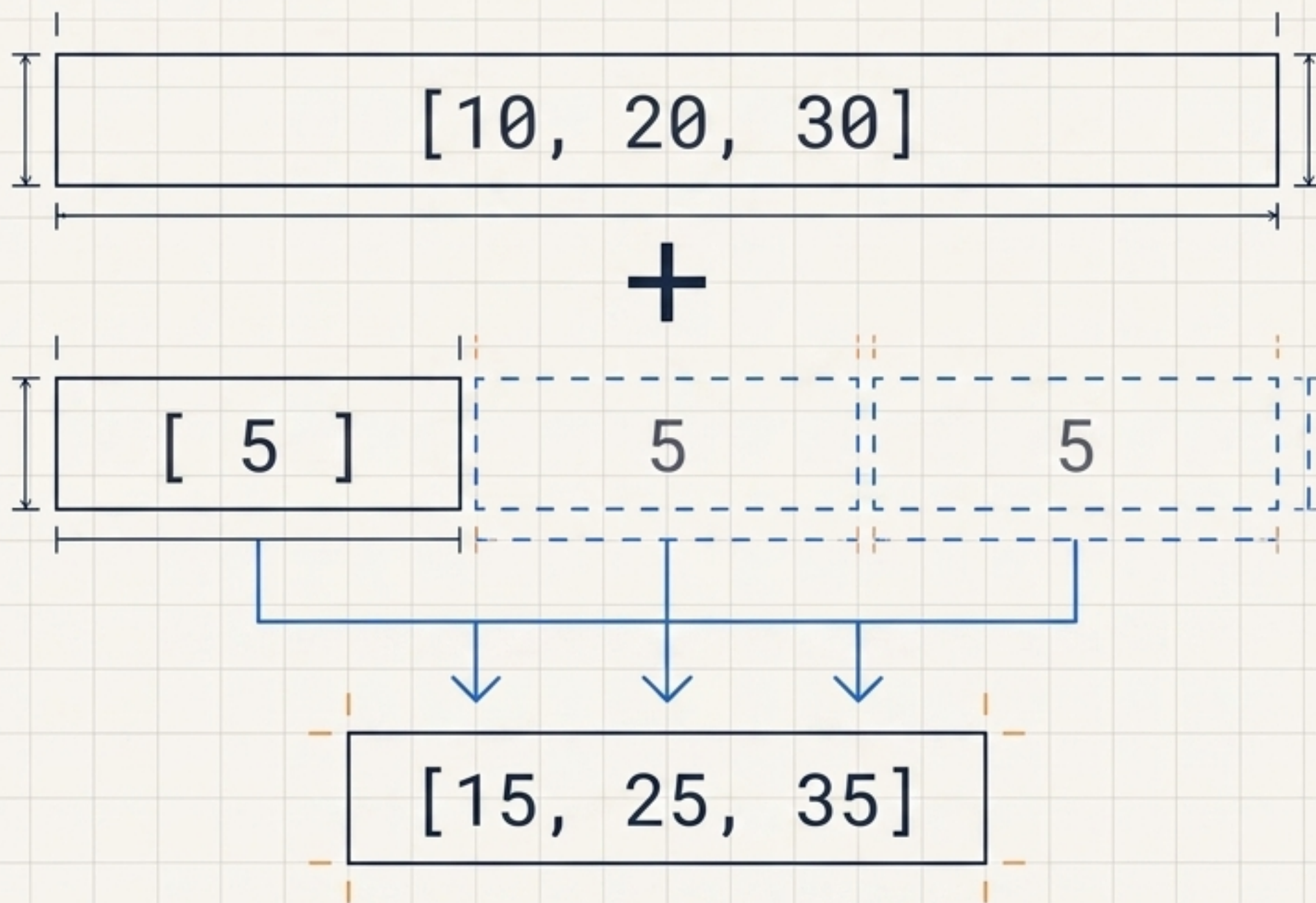
Basic mathematical operators function element-by-element across arrays of the **exact** same shape.



**IMPORTANT:** This is element-wise multiplication. It is NOT the same thing as matrix multiplication (dot product).

# The Magic of Broadcasting

NumPy can automatically perform operations between arrays of different shapes by mathematically "stretching" the smaller array without explicitly creating new copies in memory.



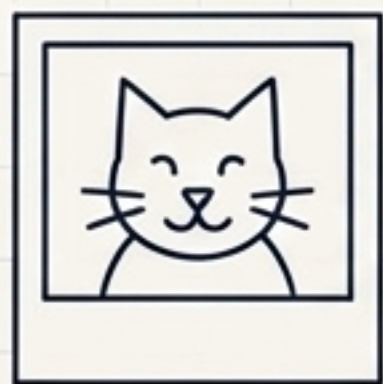
# The Bridge to Machine Learning

NumPy is the translation layer. It converts the messy, real world into pristine mathematics that neural networks can actually process.

## The Real World

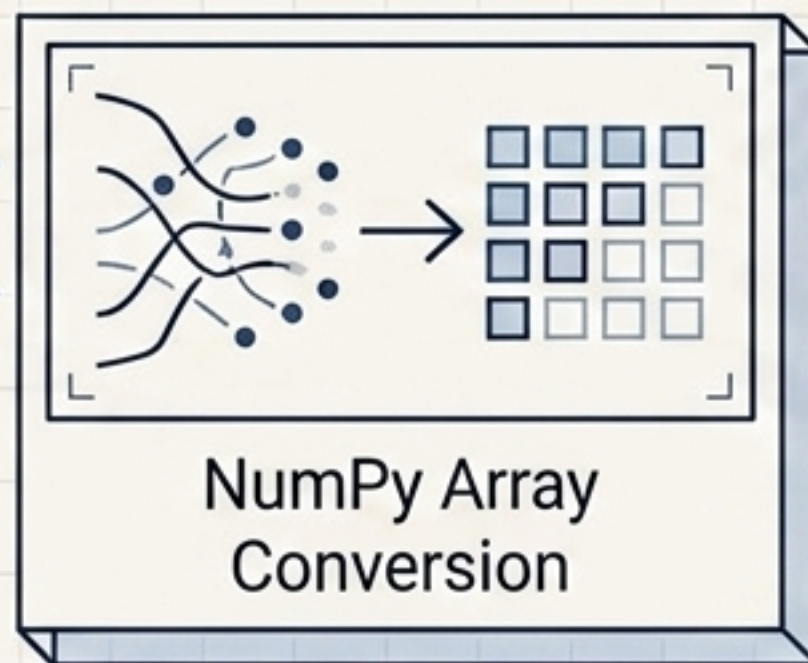
|    |   |   |     |     |
|----|---|---|-----|-----|
| .. |   |   |     |     |
| 1  | — | — | —   | —   |
| 2  | — | — | ... | —   |
| 3  | — | — |     | ... |

prices/categories



vision

## NumPy Array Conversion



The Engine

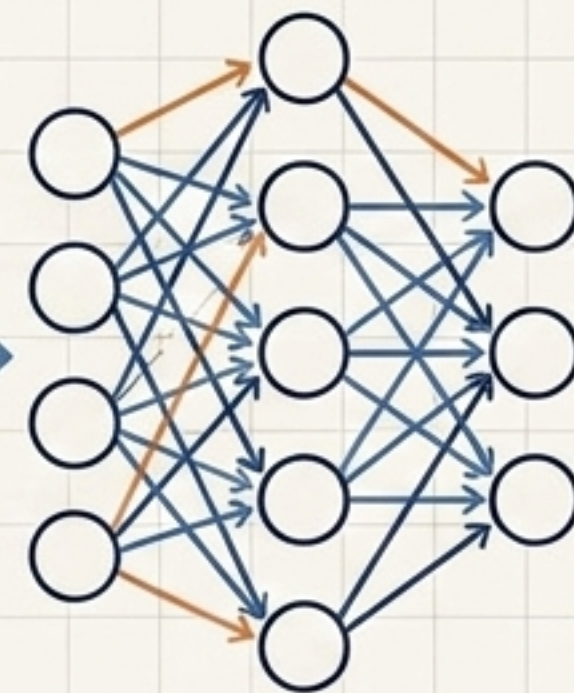
## The Math

|      |      |
|------|------|
| [1.0 | 2.0] |
| [3.0 | 4.0] |
| [5.0 | 6.0] |
| [7.0 | 8.0] |

ndarray



```
[[[1, 2], [3, 4]],  
 [[5, 6], [7, 8]]]
```



Neural Network

## The Core Mental Model

# NumPy lets Python work efficiently with numerical arrays.

1

It **shifts logic** from slow **iteration** to **parallel vectorization**.

2

It standardizes messy data into **strict geometric shapes**.

3

It builds the low-level **foundation** for **high-level AI**.

# Sandbox: Mini-Practice

## The Prompt

```
Create array [85, 90, 88, 92].  
  
Calculate average, highest, lowest.  
  
Add 5 to every score.  
  
Slice the first three scores.
```

## The Output

```
Create array [85, 90, 88, 92].  
  
Calculate average, highest, lowest.  
  
Add 5 to every score.  
  
Slice the first three scores.
```



Solve before revealing.

# The Scale of the Engine

**Python list:** “I can store numbers.”

**NumPy:** “I can calculate with them.”

**NumPy after seeing a  $10,000 \times 10,000$  matrix:**  
“Finally, some work.”

